

Computergrafik

PD Prof. Dr. rer. nat.
Marco Block-Berlitz

Sommersemester 2026



Schattierungsmodelle und visuelle Wahrnehmung

„Ohne Licht gibt es keine Kunst.“ (Rembrandt van Rijn zugeschrieben)

PD Prof. Dr. Marco Block-Berlitz

Phong-Shading

Zunächst werden die **Normalen** im Rasterisierungsprozess über das komplette Dreieck **interpoliert**. Anschließend wird die Lichtberechnung in jedem Fragment vorgenommen.

Blick auf die Retina

Die Rezeptoren in der Netzhaut sind miteinander verschaltet. Durch die **laterale Hemmung** können sich benachbarte aktive Neuronen gegenseitig abschwächen. Beispiel mit sechs Rezeptoren:

Rauhe Intensität: 10 10 10 30 30 30

Neuronenaktivität: 1 1 1 3 3 3

Wahrgenommene Intensität: 8 8 6 26 24 24

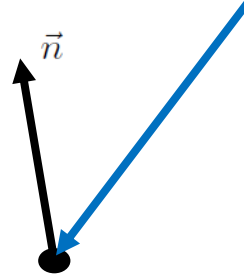
42

Ziele und Inhalt

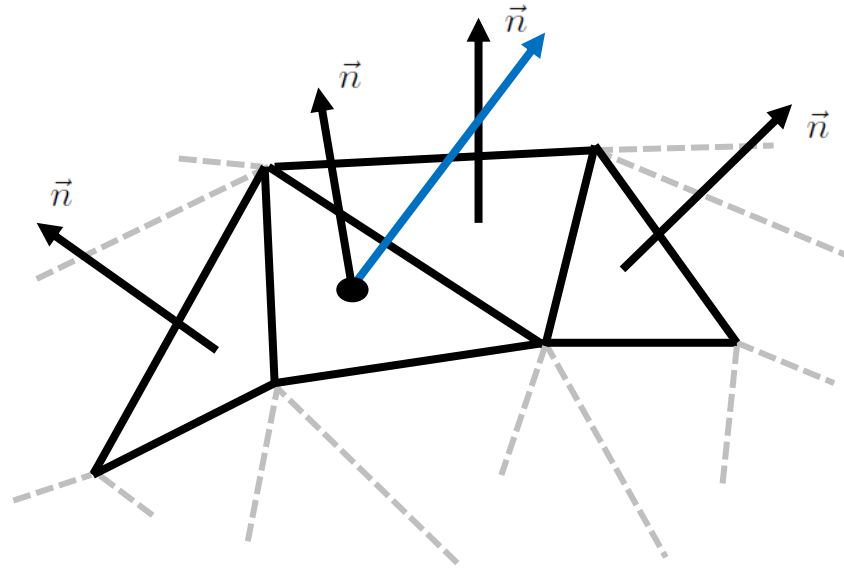
- Schattierungsmodelle
- Flat-Shading
- Gouraud-Shading
- Machsche Streifen (Mach-Band-Effekt)
- Visuelle Wahrnehmung
- Kurze Geschichte der Neuronenmodelle
- McCulloch-Pitts-Zelle
- Logische Bausteine entwerfen
- Verallgemeinerungskonzept
- Perzeptron
- Eine besondere und wichtige Funktion
- Boolesche Algebra
- Boolesche Algebra und Neuronale Netze
- Neuronales Netz als Black-Box
- Funktionskompositionen
- Backpropagation: Architektur und Ablauf
- Retinamodell mit sechs Rezeptoren
- Phong-Shading
- Flat-, Gouraud- und Phong-Shading
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



Schattierungsmodelle

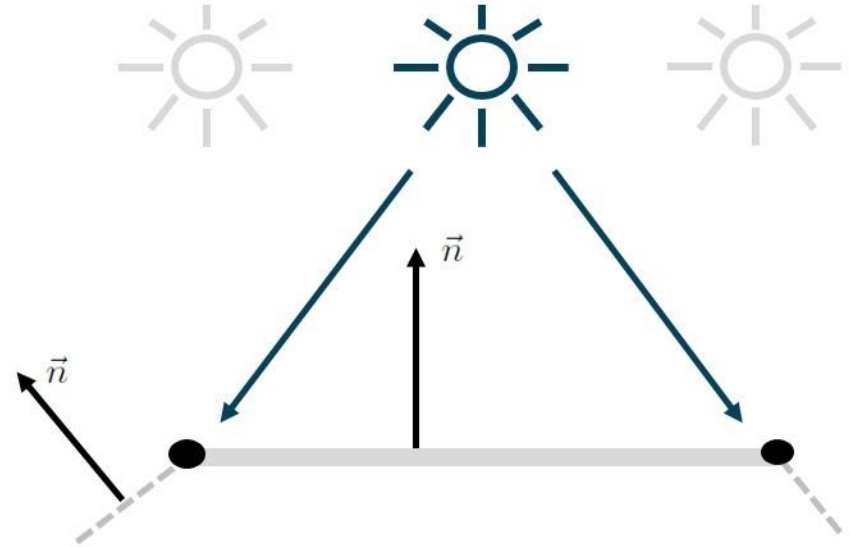
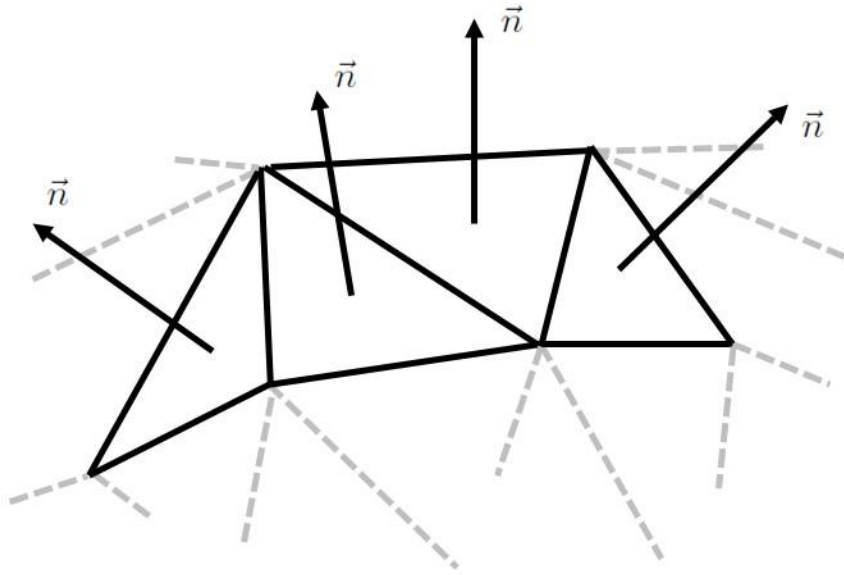


Schattierungsmodelle

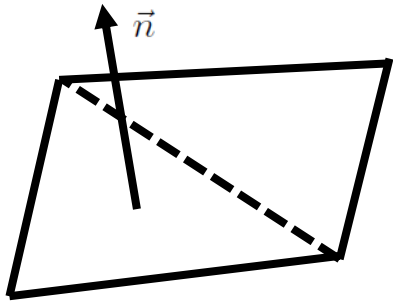
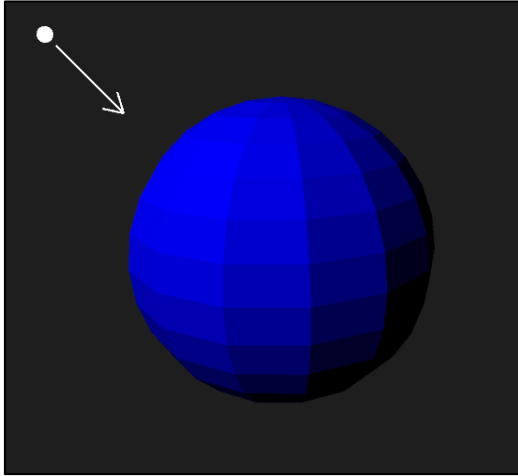


Flat-Shading

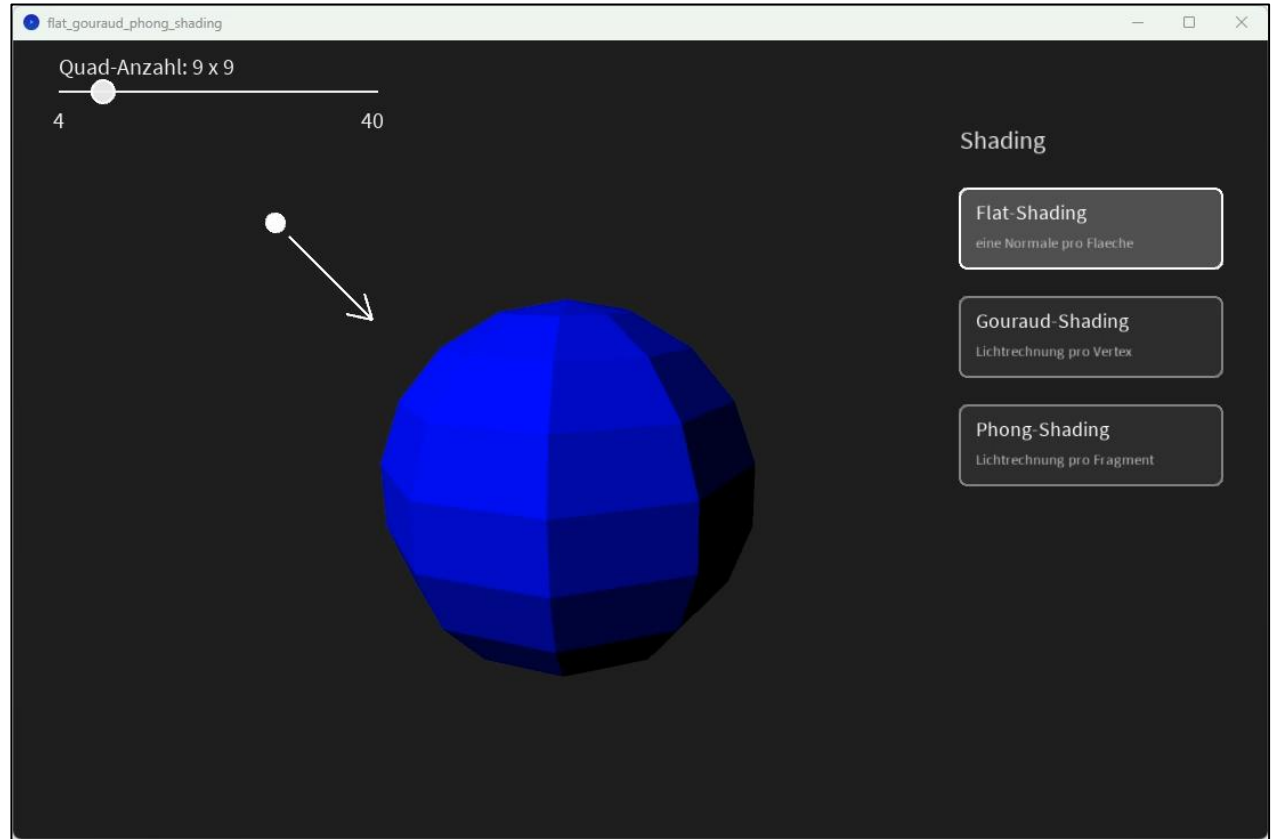
Für das gesamte Dreieck wird **eine einzige Flächennormale** verwendet. Die Beleuchtung wird genau einmal berechnet und anschließend für alle Fragmente übernommen.



Flat-Shading

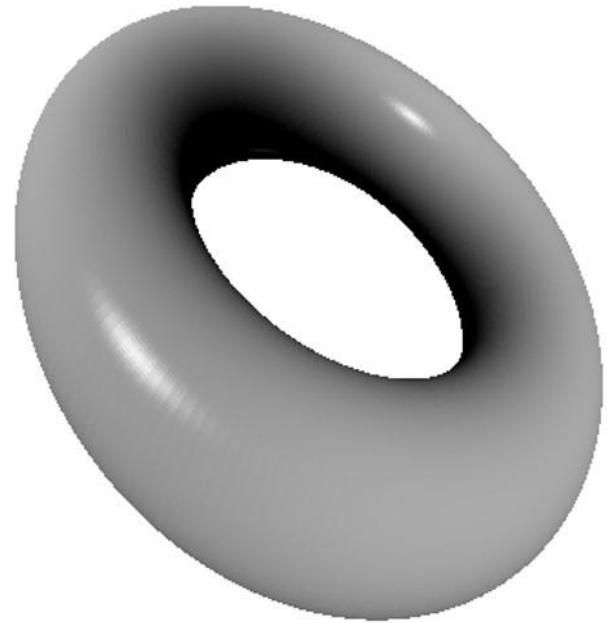
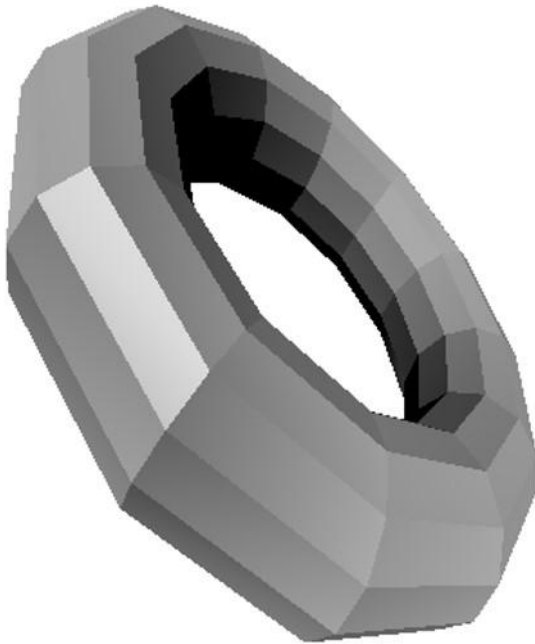


shading/flat_gouraud_phong_shading



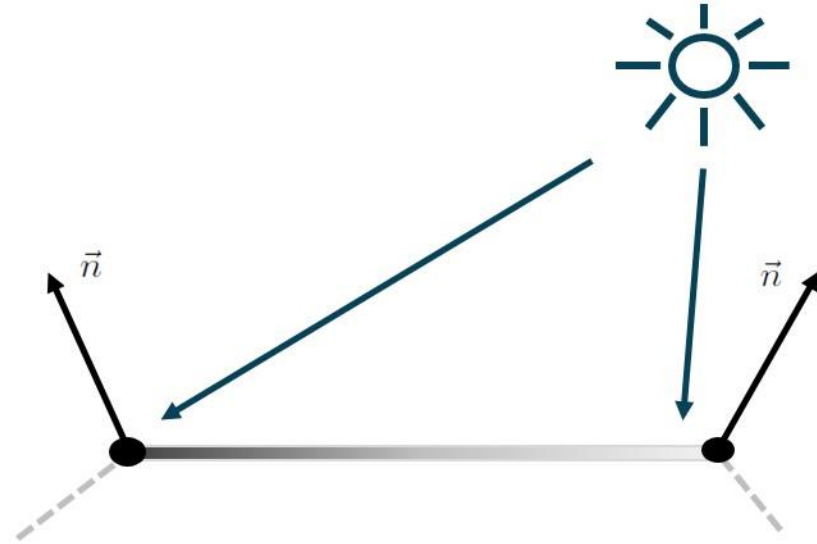
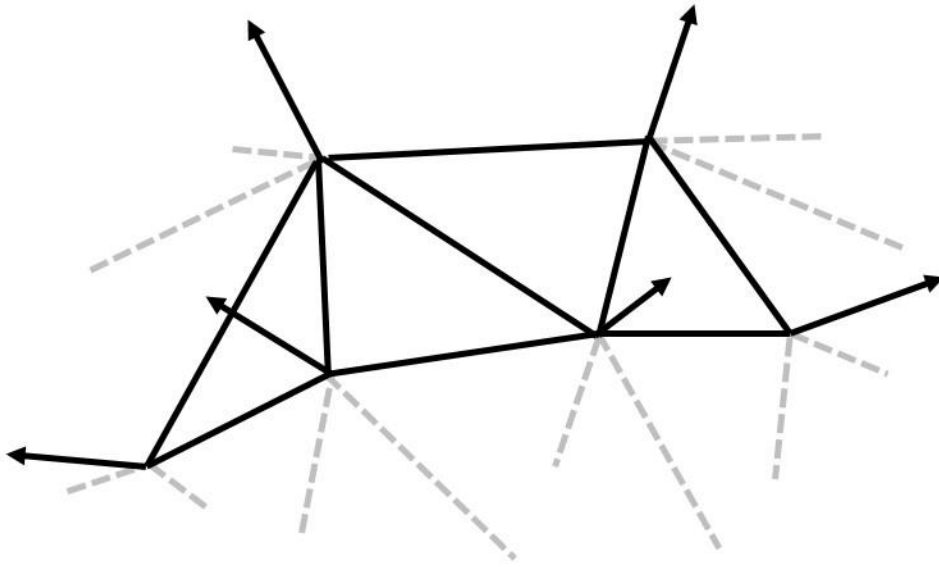
Flat-Shading und Geometriedichte

Wenn wir die Normalen über unseren [Torus mit Flat-Shading](#) verteilen, erhalten wir je nach Anzahl der Dreiecke bei geringer Anzahl sehr große einfarbige Flächen (links) oder bei einer sehr hohen Anzahl eine erheblich verbesserte Darstellung der Reflexion (rechts).

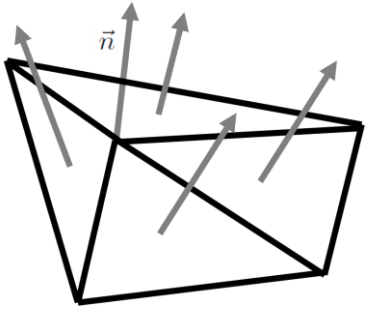
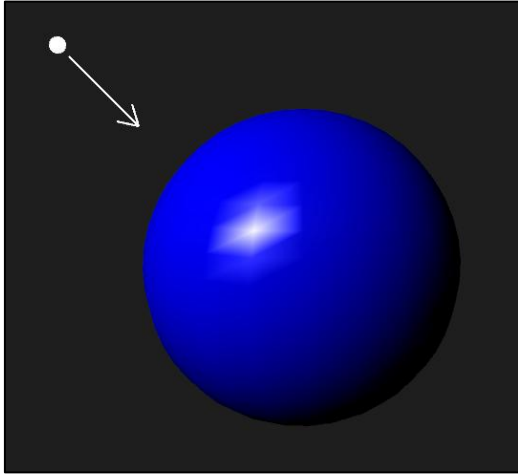


Gouraud-Shading

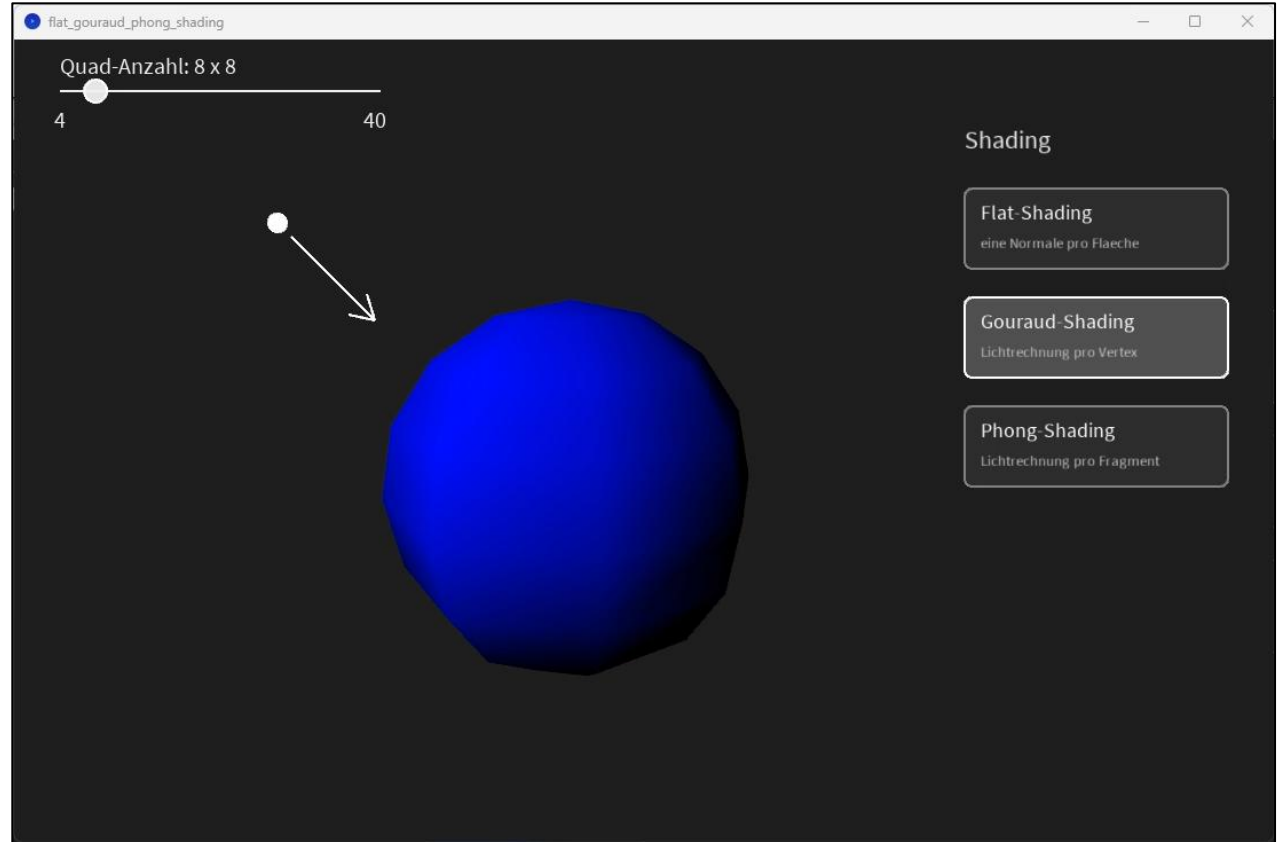
Die Lichtberechnung findet an jedem Vertex statt. Die resultierenden Farbwerte werden baryzentrisch über die Dreiecksfläche interpoliert. Spekulare Highlights können hier allerdings verloren gehen, wenn sie nicht direkt auf einem Vertex liegen.



Gouraud-Shading



shading/flat_gouraud_phong_shading

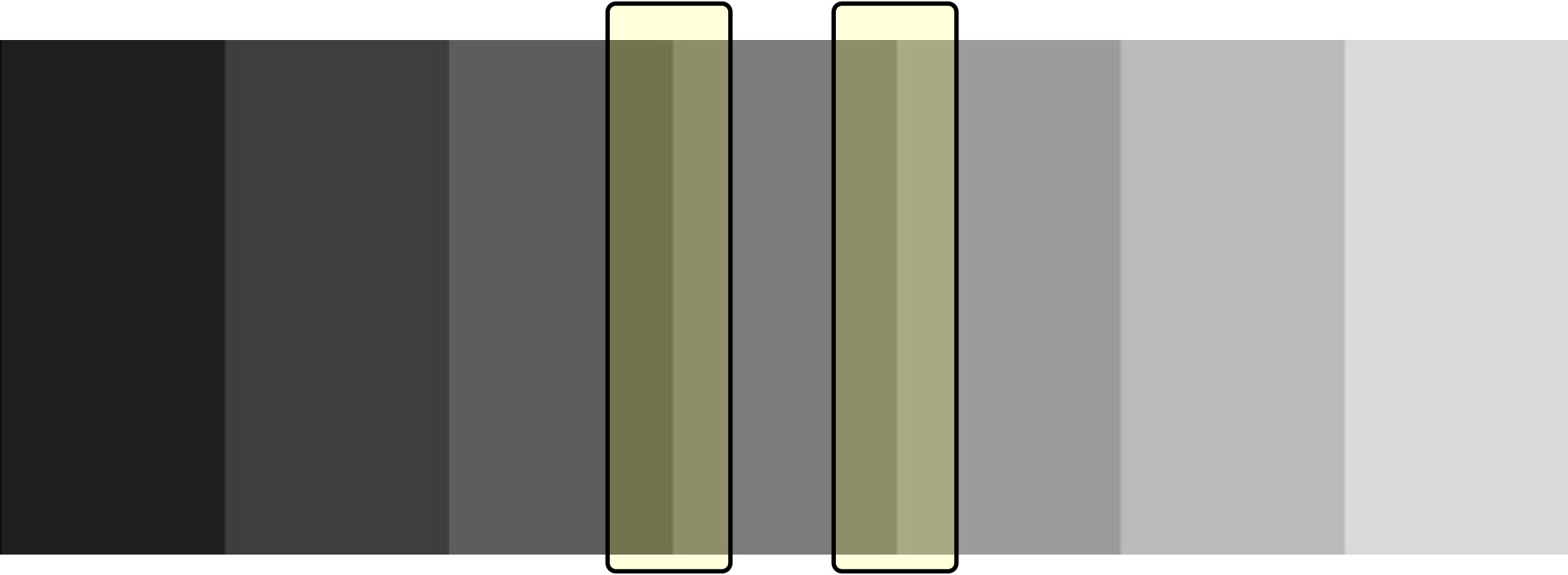




Entstehende Effekte werden hier durch eine
optische Täuschung verstärkt.

Machsche Streifen (Mach-Band-Effekt)

Bei den Übergängen von einfarbigen, grauen Flächen lassen sich entlang der Grenzen sogenannte **Machsche Streifen** als optische Illusion erkennen [1].



[1] Mach E.: „Über die Wirkung der räumlichen Vertheilung des Lichtreizes auf die Netzhaut“, In: Sitzungsberichte der mathematisch-naturwissenschaftlichen Classe der kaiserlichen Akademie der Wissenschaften. 52, S. 303–322, 1865



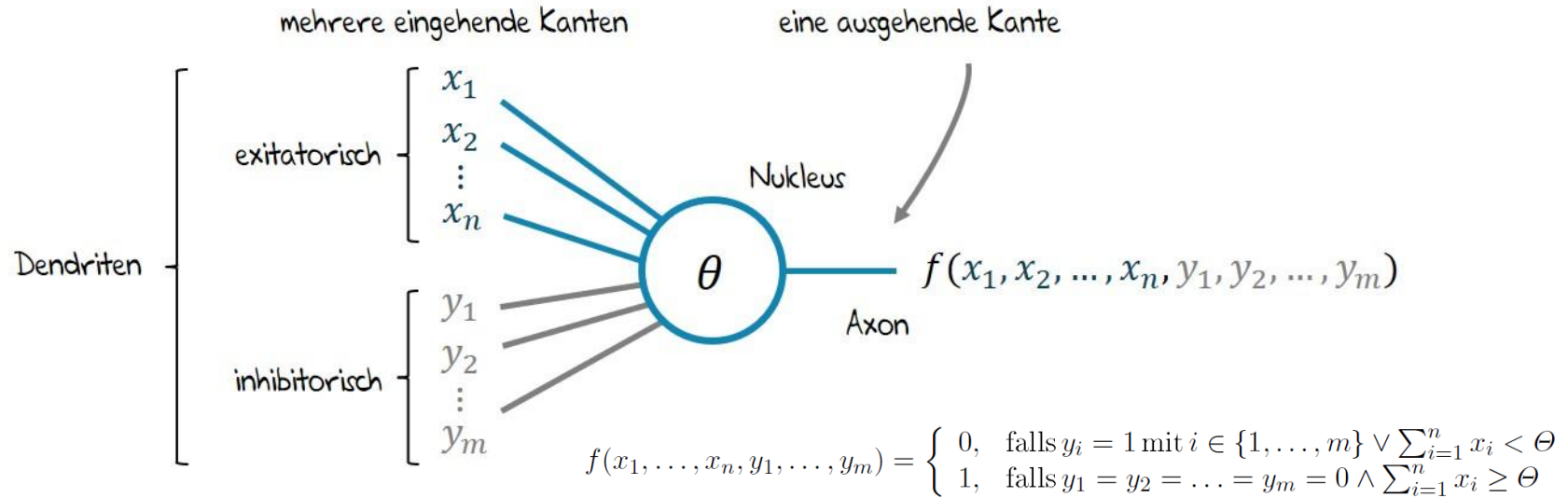
Wie funktioniert eigentlich unsere
visuelle Wahrnehmung?



Wie funktioniert eigentlich unser Gehirn?

Kurze Geschichte der Neuronenmodelle

McCulloch-Pitts hatten 1943 eine Idee, basierend auf dem biologischen Original [1]:

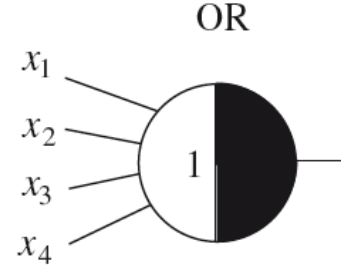
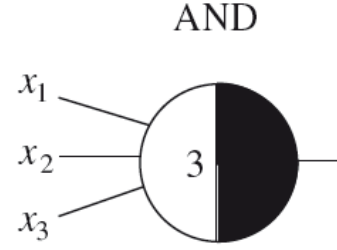
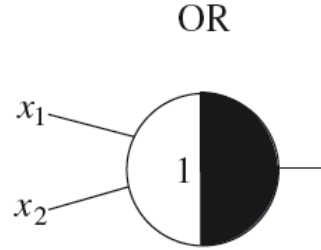
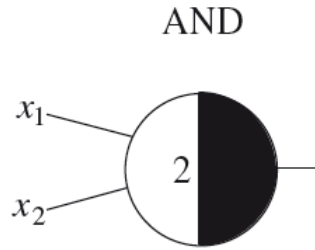




Wie können wir zeigen, dass mit der
McCulloch-Pitts-Zelle alle **booleschen Funktionen**
realisierbar sind?

Logische Bausteine entwerfen

Wir können logische Bausteine entwerfen, die eine **vollständige Basis** darstellen. Daraus lassen sich dann beliebig komplexe Funktionen entwerfen [1]:

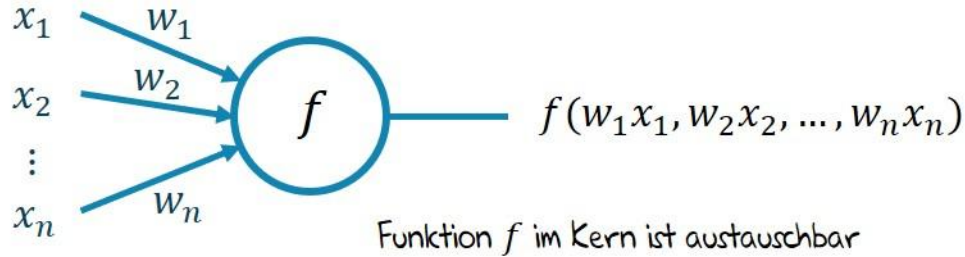


Verallgemeinerungskonzept

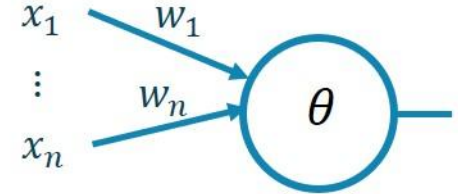
Das **Perzeptron** nach Rosenblatt stellt **eine Verallgemeinerung** der Idee dar [1]:

$$f(w_1x_1, w_2x_2, \dots, w_nx_n) = \begin{cases} 1, & \text{falls } \sum_{i=1}^n w_ix_i \geq \theta \\ 0, & \text{sonst} \end{cases} \quad f(w_1x_1, w_2x_2, \dots, w_nx_n) = \begin{cases} 1, & \text{falls } \sum_{i=1}^n w_ix_i - \theta \geq 0 \\ 0, & \text{sonst} \end{cases}$$

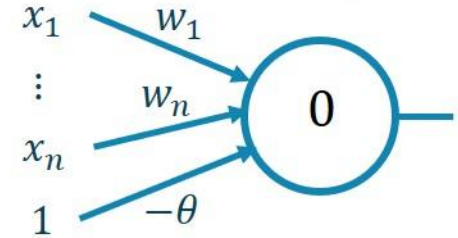
eingehende Kanten sind gewichtet



Schwellenwert im Kern

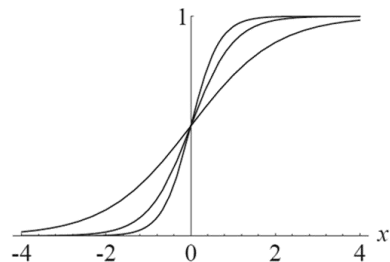


Schwellenwert als Kantengewicht



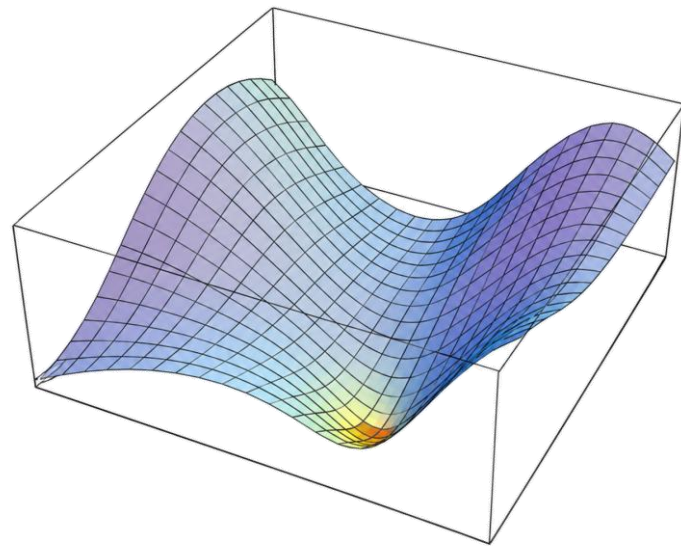
Aktivierungsfunktion: besonders und wichtig

Hinzu kommt eine stetig differenzierbare Funktion. Die [Sigmoid-Funktion](#) hat sehr angenehme Eigenschaften, sie ist stetig differenzierbar und kann je nach Bedarf der Treppenfunktion angeglichen werden. Die Ableitung ist auch sehr einfach zu berechnen:



$$s(x) = \frac{1}{1 + e^{-cx}}$$

$$s'(x) = s(x)(1 - s(x))$$



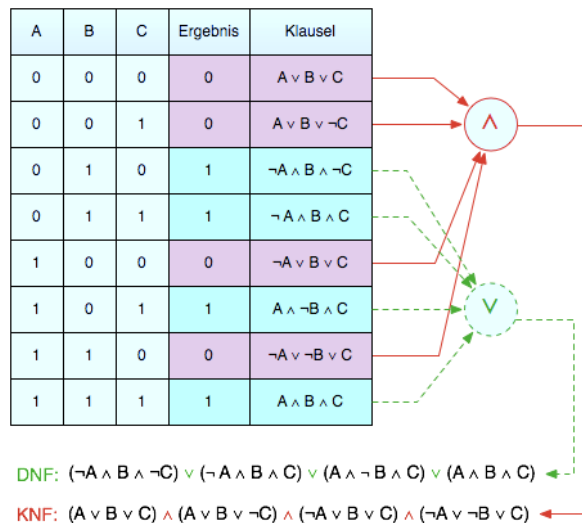
Die Sigmoidfunktion besitzt große historische Bedeutung, wird jedoch in modernen Deep-Learning-Systemen meist durch [ReLU-Varianten](#) (Rectified Linear Unit, $f(x) = \max(0, x)$) ersetzt [1].



Was kann ein **Neuronales Netz** leisten?

Boolesche Algebra

Gegeben sei eine Wertetabelle für eine 3-stellige Boolesche Funktion $f(A, B, C)$, mit den Literalen A , B und C . Eine Menge $\mathcal{B}$ von Booleschen Funktionen heißt eine **vollständige Basis**, falls jede Boolesche Funktion als Komposition von Funktionen in $\mathcal{B}$ definiert werden kann (Abbildung aus [1]).



Mit Hilfe der **disjunktiven Normalform** (Disjunktion von Konjunktionstermen) können wir alle Belegungen der Variablen angeben, die zu einer 1 führen und beschreiben die Funktion damit komplett. Analog dazu lässt sich die Funktion über die zu 0 führenden Belegungen beschreiben, in diesem Fall werden die Literale negiert und darüber die **konjunktive Normalform** (Konjunktion von Disjunktionstermen) beschrieben.

Boolesche Algebra und Neuronale Netze

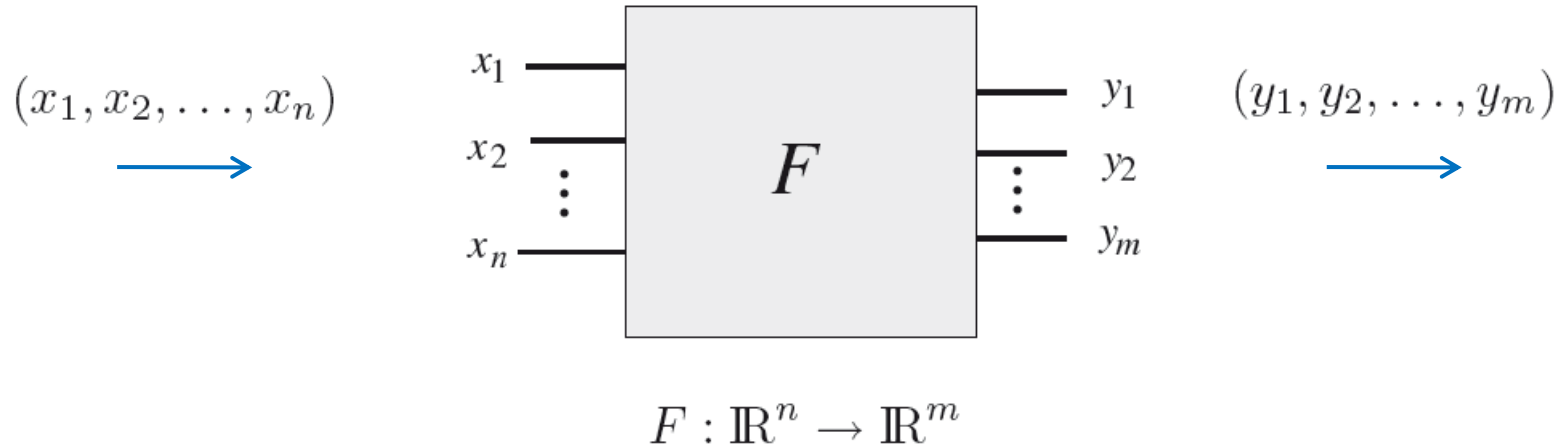
Für eine DNF/KNF benötigen wir die vollständige Wertetabelle. Angenommen, wir kennen nur ein paar Beispiele aus der Tabelle, da das Datenvolumen einfach zu groß ist. Die restlichen Funktionsergebnisse müssen wir aus den gegebenen herleiten können, also eine [Funktion approximieren](#).

a	b	c	d	e	f	g	h	f(a, b, c, d, e, f, g, h)
1	0	0	1	1	0	1	0	1
0	0	1	0	1	1	1	0	1
0	0	0	0	0	0	0	1	0
...								...
1	0	0	0	0	0	1	1	0

256 mögliche
Belegungen und mit
jedem neuen Literal
verdoppelt sich die
Anzahl

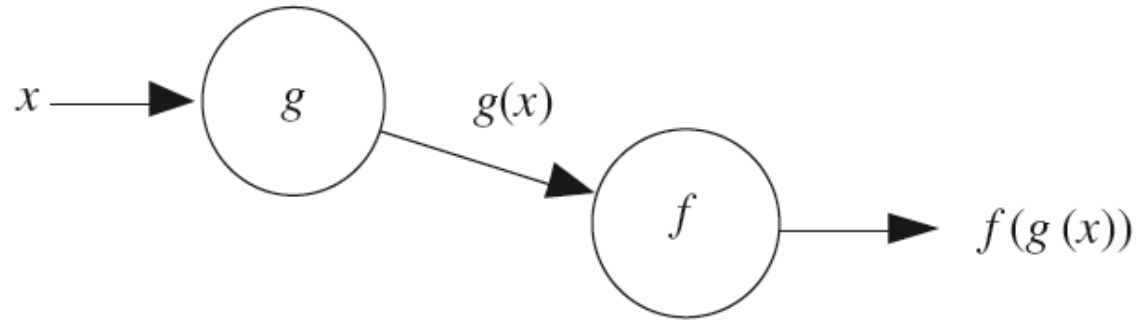
Neuronales Netz als Black-Box

Wir codieren „Anfragen“ als Vektoren und „Wissen“ ebenfalls als Vektoren [1]:



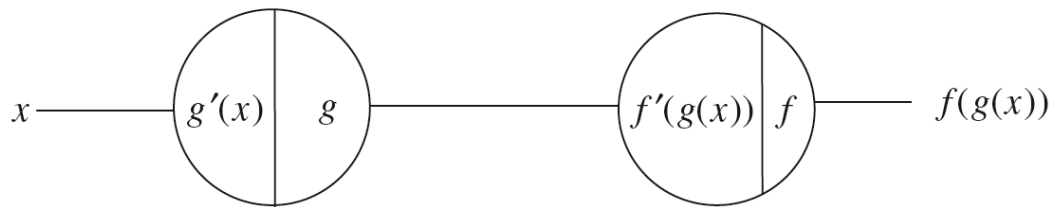
Funktionskompositionen

Wir können in die eine Richtung Funktionen verknüpfen und in die andere Richtung sogar Funktionen automatisch ableiten!

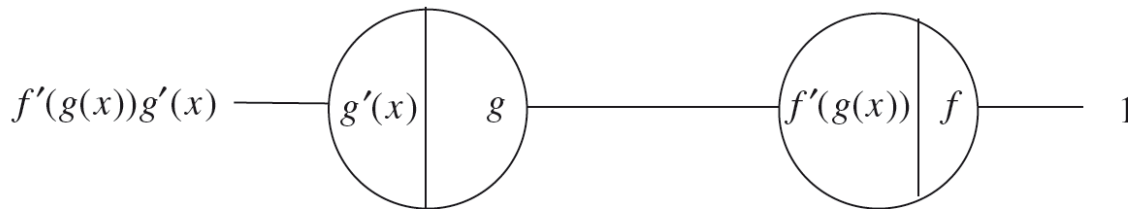


Funktionskompositionen: Kettenregel

Von links nach rechts erhalten wir eine **Funktionskomposition**. Dabei speichern wir die lokalen Ergebnisse und können die Ableitung sofort angeben:

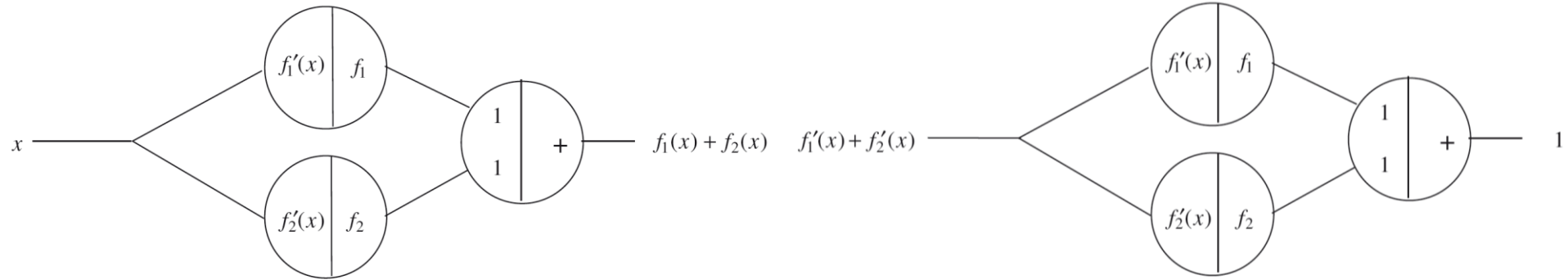


Wenn wir jetzt mit einem eingehenden Gradienten mit dem Wert **1** von rechts nach links durch das Netz gehen und die lokalen Ableitungen aufmultiplizieren, erhalten wir die **Gesamtableitung**. Das Ergebnis ist erstaunlich, innere Ableitung mal äußere Ableitung (Kettenregel), ganz wie in der Schule:



Funktionskompositionen: Addition

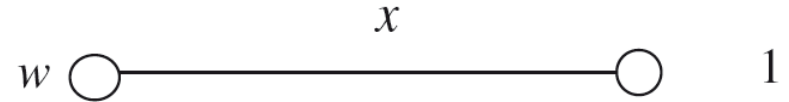
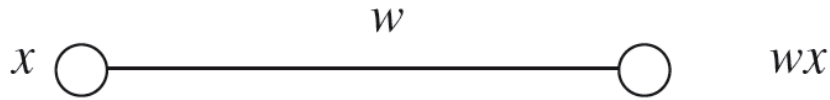
Schauen wir uns als nächstes die **Addition zweier Funktionen** im Feedforward-Schritt an. Wir erhalten also $f_1(x) + f_2(x)$. Auch hier wieder die Frage, ob wir die Ableitung durch die Speicherung der lokalen Ableitungen automatisch ermitteln können.



Auch hier ist die Antwort: Ja!

Funktionskompositionen: Nicht nach Eingabe sondern Gewicht

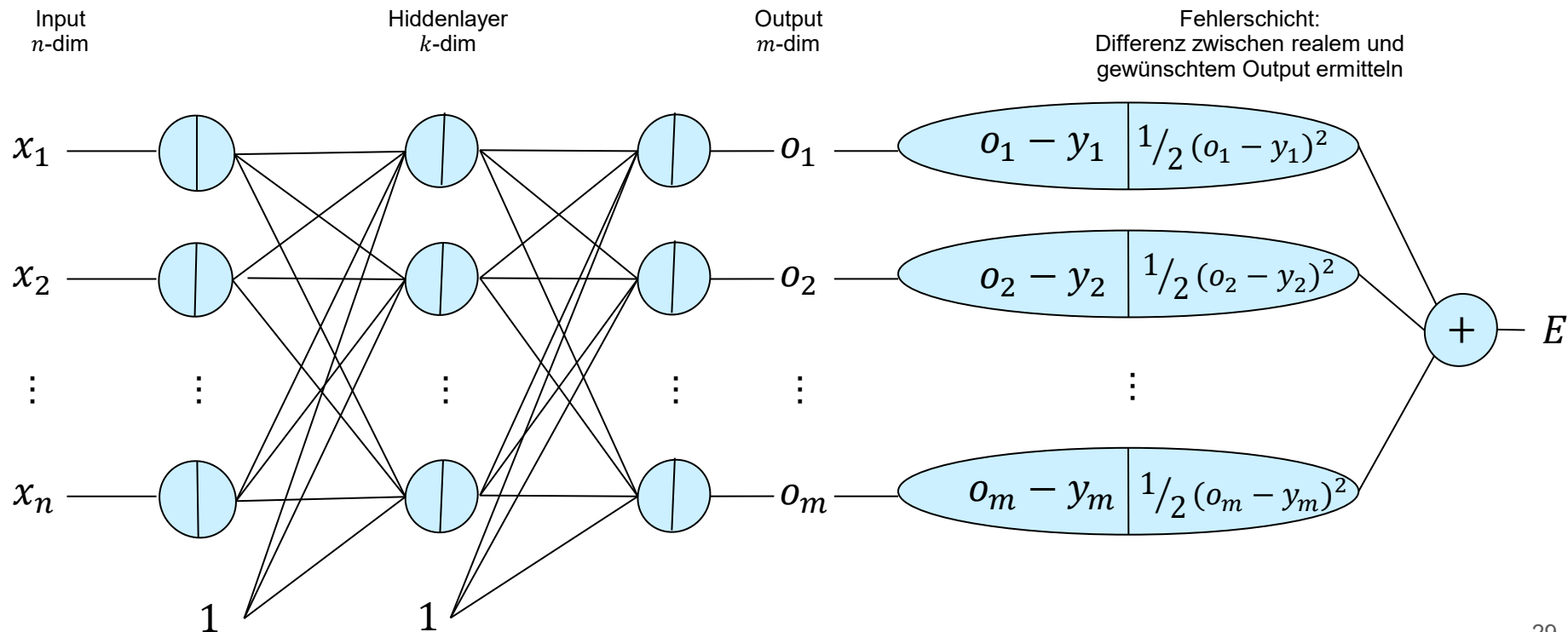
Wir haben noch ein Problem: **Nicht die Eingabe wollen wir ändern, sondern das Gewicht**. Wir haben in unseren Vorüberlegungen allerdings nach der Eingabe abgeleitet ...



Glücklicherweise ist die Multiplikation kommutativ, wir können also x und w vertauschen! Damit ist x das Kantengewicht und wir können nach w ableiten.

Backpropagation: Architektur und Ablauf

In der Trainingsphase liegen [Anfrage-Antwort-Paare](#) vor. Der Fehler wird berechnet und zur Anpassung der Gewichte durch [Backpropagation](#) verwendet:



Blick auf die Retina

Die Rezeptoren in der Netzhaut sind miteinander verschaltet. Durch die **laterale Hemmung** können sich benachbarte, aktive Neuronen gegenseitig abschwächen. Beispiel mit sechs Rezeptoren:

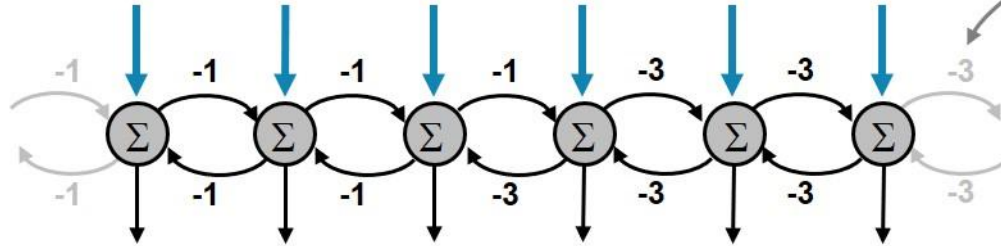
$$p = \frac{1}{10}$$

Reale Intensität

C :

10 10 10 30 30 30

Neuronenschicht



hemmende Kante

Wahrgenommene Intensität

C'
 C''

8 8 6 26 24 24

10 10 7,5 32,5 30 30

0 0 2,5 -2,5 0 0

$$\begin{aligned} C' &= C - 2pc \\ &= C(1 - 2p) \\ &= C\left(1 - \frac{2}{10}\right) \\ &= C \cdot \frac{8}{10} \end{aligned}$$

Ausgleich mit

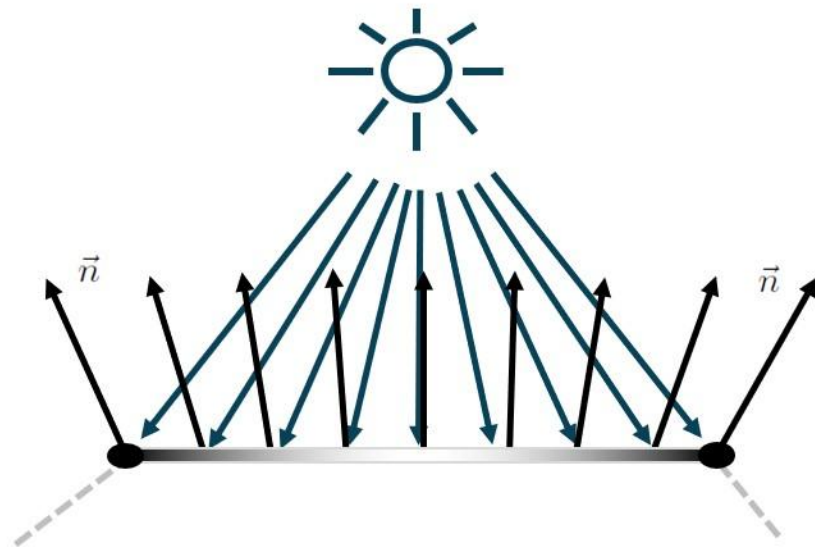
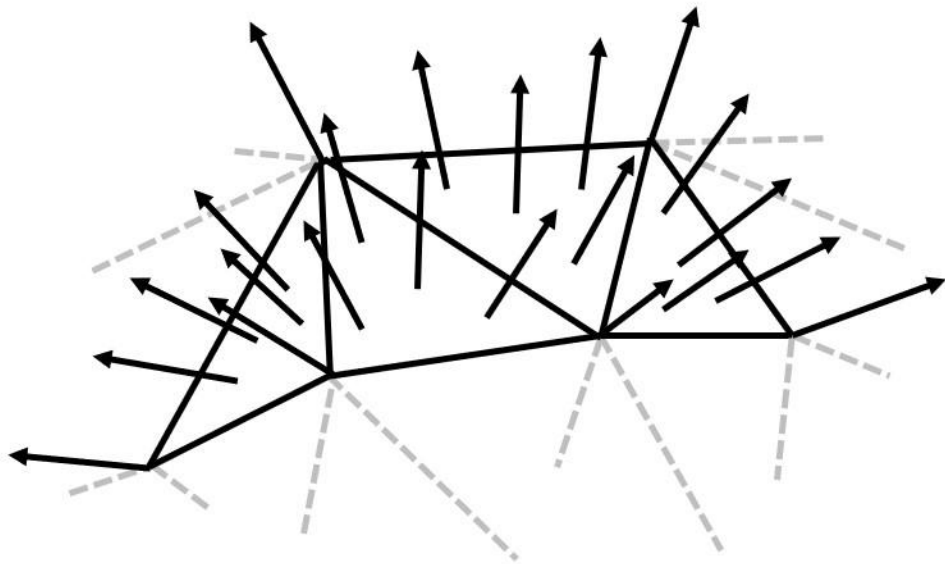
$$\frac{1}{1-2p} = \frac{1}{1-\frac{2}{10}} = \frac{1}{\frac{8}{10}} = \frac{10}{8}$$



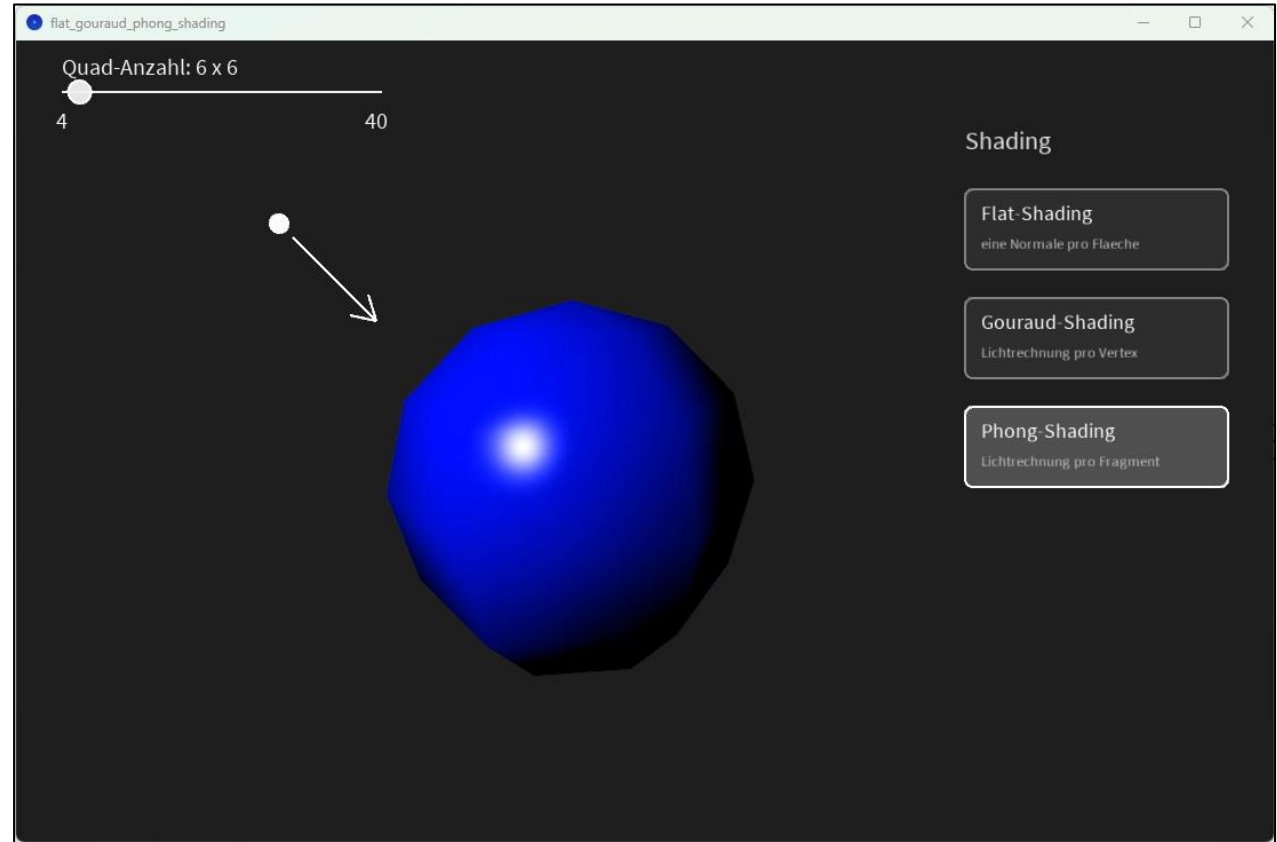
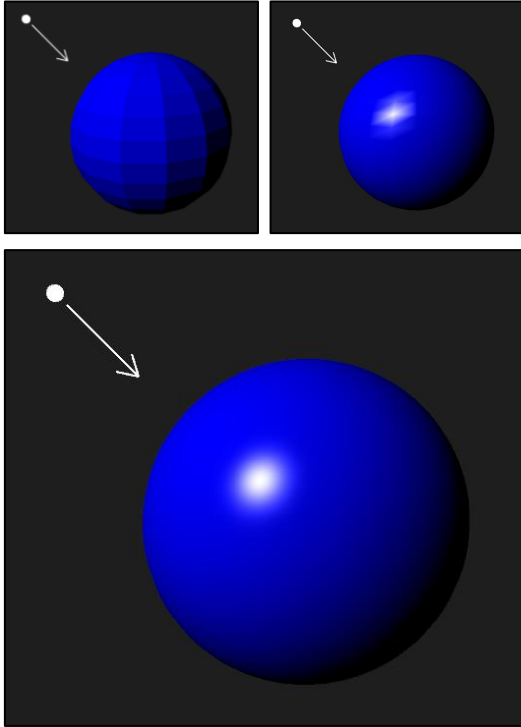
Aus diesem Grund müssen wir
Diskontinuitäten vermeiden

Phong-Shading

Zunächst werden die **Normalen** im Rasterisierungsprozess über das komplette Dreieck **interpoliert**. Anschließend wird die Lichtberechnung in jedem Fragment vorgenommen.



Phong-Shading

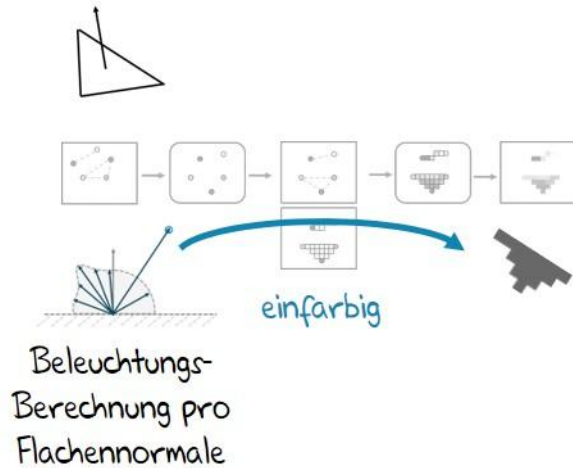


shading/flat_gouraud_phong_shading

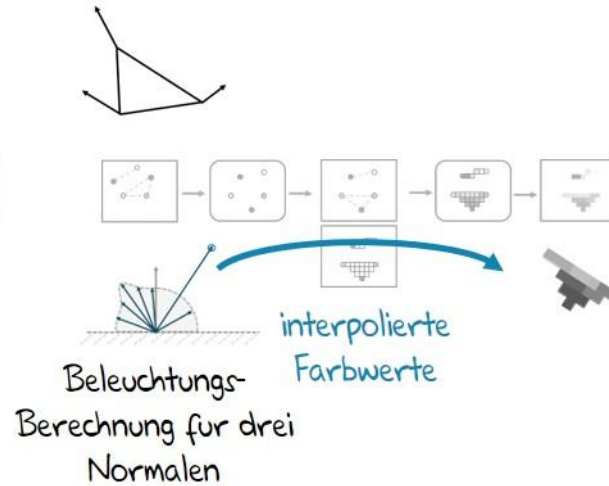
Flat-, Gouraud- und Phong-Shading

Mit dem Flat-Shading haben wir zwar das deutlich schnellste Verfahren, da die Beleuchtungsrechnung nur an einer Stelle stattfindet, aber damit auch die qualitativ einfachste Darstellung.

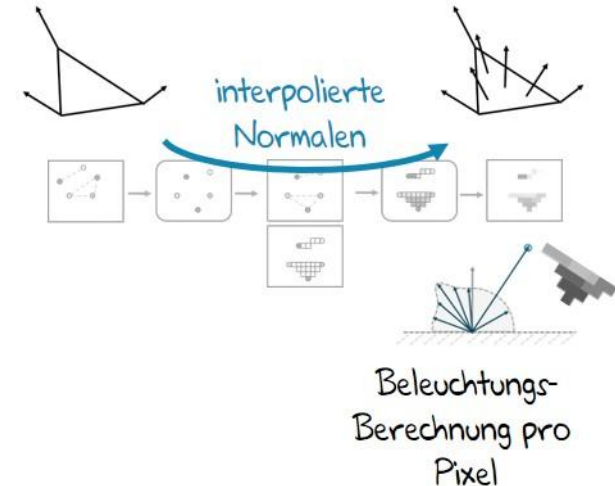
Flat-Shading



Gouraud-Shading



Phong-Shading



Elementare Beleuchtungsmodelle

Key Takeaways

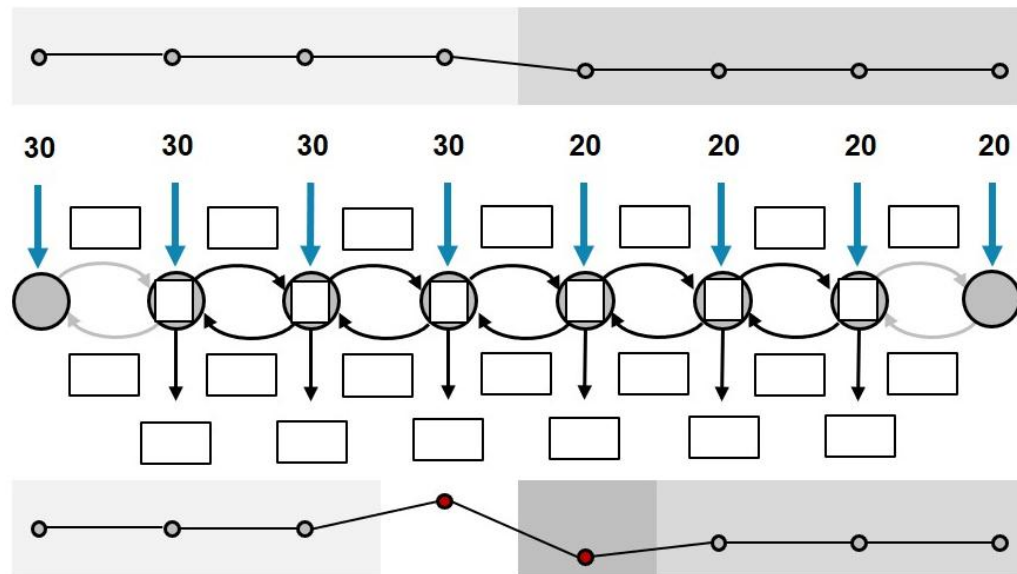


- Flat-Shading verwendet eine Normale pro Polygon
- Gouraud-Shading interpoliert Farbwerte zwischen Vertices
- Phong-Shading interpoliert Normalen und berechnet Licht pro Fragment
- Mach-Bänder entstehen durch laterale Hemmung in der Netzhaut und verstärken die Kantenwahrnehmung
- Die menschliche Wahrnehmung beeinflusst die Bewertung der Bildqualität

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

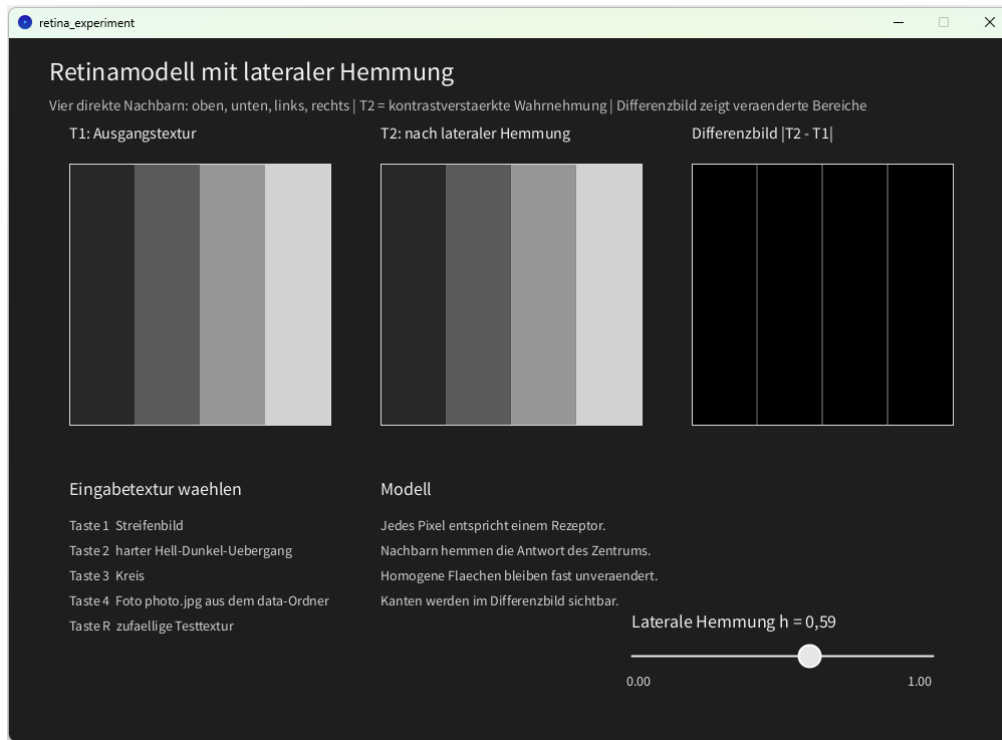
- (1) Beschreiben Sie kurz die Unterschiede zwischen Flat-, Gouraud- und dem Phong-Shading.
- (2) Erläutern Sie das Phänomen des Machband-Effekts durch die Verarbeitung der Neuronen in Bezug auf Schattierungsmodelle anhand des konkreten Beispiels mit sechs Neuronen und geben Sie die 26 Lücken vollständig an:



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) Simulieren Sie analog zum Beispiel (Retina mit 6 Neuronen auf Folie 31) ein zweidimensionales Feld von Retinaneuronen. Jedes Neuron soll dabei maximal mit seinen direkten vier Nachbarn (oben, unten, links, rechts) verschaltet sein. Die laterale Hemmung der Nachbarn soll zunächst $\frac{1}{10}$ betragen, aber für spätere Tests regelbar sein. Für eine gegebene Textur T_1 soll ein Pixel einem Neuron entsprechen. Schreiben Sie ein Programm, das eine zweite Textur T_2 mit der Anwendung des Machband-Effekts erzeugt. Anschließend soll das Differenzbild aus T_2 und T_1 visualisiert werden.





Vielen Dank für die Aufmerksamkeit